Jordan Pauzie & Julian Savini
CS: 151-2 Special Topics in Data Infrastructure and Systems
Professor Lillethun & Professor Sundar
10/22/25

Project 2 Beta Writeup

Design Overview:

Our initial plan of action was to implement the projection algorithm for render() and then optimize smaller portions of serial code such as invariant color calculations and symmetry of gravitational force. The high-level idea of our projection algorithm as to sort all the spheres by distance to the eye, filter out all the spheres whose centers weren't within the scope of the projection plane (to reduce the number of potential calculations), construct bounding boxes around the 2D circular images of the spheres (because from the perspective of the eye and projection plane, every sphere appears to be a circle), and then attempt to map points from the projection plane to the points on the sphere (within the restricted bounds). We would then maintain a 1D "filled" array which keeps track as to whether a particular point on the projection plane had already been colored or not; this would allow us to avoid iterating over already-filled pixels and save computation time. However in our initial implementation of the projection algorithm, there were several issues. Firstly, spheres on the edges of the plane were being omitted and secondly, their 2D circular images were being cut off due to the bounds being too tight (and not centered for some reason). We fixed the edge-sphere issue by computing bounding boxes for every sphere and then bounded those coordinates by the coordinates of the projection plane, and the tight bound issue by doubling the size of the bounds (as a temporary fix). In doing so, our new design was able to achieve tier 24 on ./tiers.

We then attempted to handle precomputing color invariants by augmenting the sphere struct but ran into the issue of the render() function declaring all spheres passed in as static; this meant that I wouldn't be able to store invariant data within the sphere, and proved to be a major stumbling block. Instead, we decided to obtain a speed-up by simply parallelizing render() using OpenCilk. Two areas of logic were parallelized: the zero-initialization of the filled[] array used to keep track of colored pixels, and the double for-loop which handled coloring pixels. Since the actual computation of red, green, blue relied on information unique to that particular pair of x and y coordinates, each pixel-coloring was functional independently of one another. This meant that parallelizing the outer for-loop was completely safe from data dependencies, and performance was improved significantly. These optimizations were further built upon by our team's optimization of simulate.c where we focused on exploiting symmetry.

Exploiting symmetry involves understanding Newton's third law. The same pull i has on j, is equal to j's pull i on, just as a negative value. Instead of computing the forces twice, we compute each pair of forces one time. If there are n spheres, whereas it would take n^2 calculations, it is now n^2 / 2. All pairwise forces are computed into a cache, and then they are stacked in the same order as the original algorithm, so it can pass correctness tests. For situations where spheres have very large masses, we decided to use the original algorithm. This is because although the gifs look accurate, we assume there are floating point miscalculations, which are thus failing correctness. We parallelized update_accelerations_libstudent() by adding cilk_for to

the accumulation loop. This loop only reads from force_cache and writes to spheres[i + n_spheres].accel, so there are no data races.

General State of Completeness:
At the moment, the general, high-level state of our algorithm feels solid, however there are many improvements to performance that can be made. Our ideas will be described in further detail in the next section, but we want to debug the non-centered bounds of our spheres, implement more serial optimizations, and parallelize our code where apt. We were not aware of any bugs in correctness in this submission. However, we had to run ./bin/find-tier to determine we reached tier 46. When running ./scripts/test_all, it looks like the tiers are not able to run, because of some process cilksan is doing, so that will be investigated for the final submission. Running ./scripts/test_simulations or ./scripts/test_all proves the correctness of our code.

Ideas for the Final Submission:
- The bounding boxes of the spheres are still not centered, so we need to look into that.
- Vectorization using AVX intrinsics and Clang __builtin functions could immensely speed up runtime.
- Efficient collision detection has not been looked over yet. This will be done in the final.
- Precomputing color invariant calculations is an interesting idea, but our initial implementation had issues with augmenting the sphere struct when render() declares them as static.

Additional Information & Acknowledgements:
- Julian spent quite some time on a method to take the center of the sphere and its radius, get coordinate pixels for the 8 corners of a square, and then broadcast those points onto the 2D plane. This ultimately did not work, but it is worth sharing a failed implementation.

Tier 2 Performance Tracking:

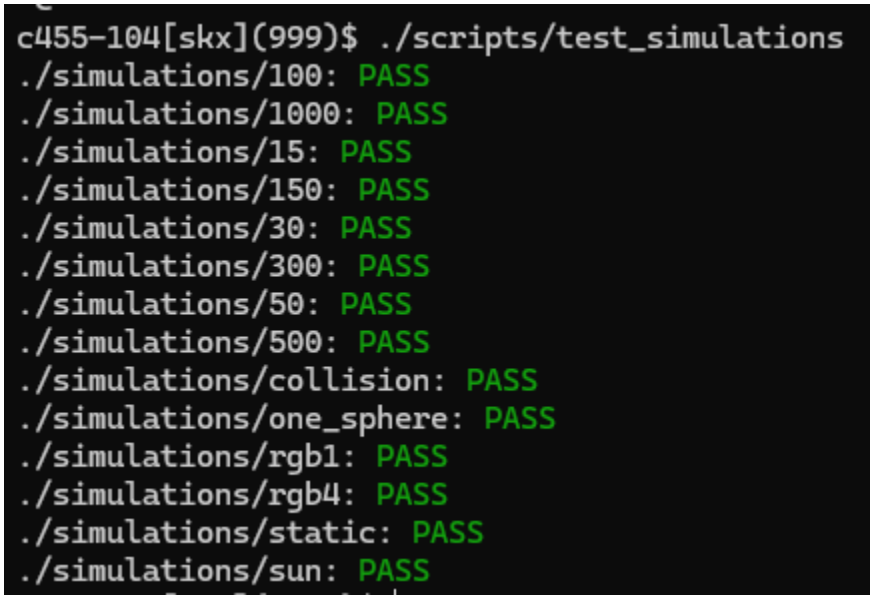
Version	**Tier**	**Changes (if any)**
Benchmark	30	
Original	0	
Version 1	24	Replaced ray tracing algorithm with projection algorithm
Version 2	32	Exploited symmetry w/ cilk
Version 3	47	Parallelized render() function

Project Log:
Important Note: We met and worked together on all parts of the project.

#	**Date**	**Duration**	**Description**
1	October 15	2	Carefully reading instructions, understanding problem space,

			reading files
2	October 15	1	Implementation of 8 corners to capture planet, failed implementation
3	October 15	1	Partial Implementation of 3D projection algorithm
4	October 17	6	Debugging 3D projection algorithm
5	October 20	2	Exploiting Symmetry, nearly correct implementation
6	October 20	2	Color invariance
7	October 22	.5	Looking current work, strategizing until stampede3 is up
8	October 22	1	Write up
9	October 22	3	Exploiting Symmetry Fixed, implemented standard calc for larger spheres
10	October 23	1	Parallelized acceleration function + merging code
11	October 23	1	Write up
12	-		-

Screenshots:
Below are a couple of snapshots showing our success in idev:



```
c455-104[skx](997)$ ./bin/find-tier
REMINDER: This is only timing, NOT CORRECTNESS. Ensure you also run correctness tests
PASS (ayy!):    Tier 0 :        Ran 320x320     image with 800 bodies in 37ms (s: 20, r: 15, is: 0, ir: 0)
PASS (yeee!):   Tier 1 :        Ran 320x320     image with 832 bodies in 29ms (s: 12, r: 13, is: 0, ir: 0)
PASS (skrrt!):  Tier 2 :        Ran 336x336     image with 865 bodies in 29ms (s: 15, r: 12, is: 0, ir: 0)
PASS (boyah!):  Tier 3 :        Ran 352x352     image with 899 bodies in 33ms (s: 15, r: 15, is: 0, ir: 0)
PASS (yeee!):   Tier 4 :        Ran 368x368     image with 935 bodies in 37ms (s: 15, r: 18, is: 0, ir: 0)
PASS (yay!):    Tier 5 :        Ran 384x384     image with 973 bodies in 38ms (s: 18, r: 18, is: 0, ir: 0)
PASS (ayy!):    Tier 6 :        Ran 400x400     image with 1012 bodies in 43ms (s: 18, r: 21, is: 0, ir: 0)
PASS (woot!):   Tier 7 :        Ran 416x416     image with 1052 bodies in 44ms (s: 21, r: 21, is: 0, ir: 0)
PASS (boyah!):  Tier 8 :        Ran 432x432     image with 1094 bodies in 50ms (s: 24, r: 25, is: 0, ir: 0)
PASS (eoo!):    Tier 9 :        Ran 448x448     image with 1138 bodies in 55ms (s: 24, r: 28, is: 0, ir: 0)
PASS (boyah!):  Tier 10 :       Ran 464x464     image with 1184 bodies in 57ms (s: 27, r: 28, is: 0, ir: 0)
PASS (yeee!):   Tier 11 :       Ran 480x480     image with 1231 bodies in 62ms (s: 30, r: 30, is: 0, ir: 0)
PASS (yay!):    Tier 12 :       Ran 512x512     image with 1280 bodies in 68ms (s: 30, r: 34, is: 0, ir: 0)
PASS (boyah!):  Tier 13 :       Ran 528x528     image with 1332 bodies in 73ms (s: 34, r: 36, is: 0, ir: 0)
PASS (yeee!):   Tier 14 :       Ran 544x544     image with 1385 bodies in 79ms (s: 36, r: 40, is: 0, ir: 0)
PASS (eoo!):    Tier 15 :       Ran 576x576     image with 1440 bodies in 82ms (s: 39, r: 40, is: 0, ir: 0)
PASS (yay!):    Tier 16 :       Ran 592x592     image with 1498 bodies in 93ms (s: 42, r: 47, is: 0, ir: 0)
PASS (woot!):   Tier 17 :       Ran 608x608     image with 1558 bodies in 98ms (s: 48, r: 49, is: 0, ir: 0)
PASS (ayy!):    Tier 18 :       Ran 640x640     image with 1620 bodies in 108ms (s: 51, r: 56, is: 0, ir: 0)
PASS (eoo!):    Tier 19 :       Ran 672x672     image with 1685 bodies in 114ms (s: 54, r: 57, is: 0, ir: 0)
PASS (yay!):    Tier 20 :       Ran 688x688     image with 1752 bodies in 124ms (s: 60, r: 61, is: 0, ir: 0)
PASS (yay!):    Tier 21 :       Ran 720x720     image with 1823 bodies in 132ms (s: 63, r: 66, is: 0, ir: 0)
PASS (woot!):   Tier 22 :       Ran 752x752     image with 1895 bodies in 148ms (s: 70, r: 76, is: 0, ir: 0)
PASS (boyah!):  Tier 23 :       Ran 784x784     image with 1971 bodies in 157ms (s: 75, r: 79, is: 0, ir: 0)
PASS (eoo!):    Tier 24 :       Ran 816x816     image with 2050 bodies in 175ms (s: 81, r: 91, is: 0, ir: 0)
PASS (skrrt!):  Tier 25 :       Ran 848x848     image with 2132 bodies in 188ms (s: 88, r: 97, is: 0, ir: 0)
PASS (boyah!):  Tier 26 :       Ran 880x880     image with 2217 bodies in 199ms (s: 96, r: 101, is: 0, ir: 0)
PASS (boyah!):  Tier 27 :       Ran 912x912     image with 2306 bodies in 216ms (s: 102, r: 109, is: 0, ir: 0)
PASS (yeee!):   Tier 28 :       Ran 944x944     image with 2398 bodies in 234ms (s: 112, r: 120, is: 0, ir: 0)
PASS (yay!):    Tier 29 :       Ran 992x992     image with 2494 bodies in 254ms (s: 120, r: 130, is: 0, ir: 0)
PASS (eoo!):    Tier 30 :       Ran 1024x1024   image with 2594 bodies in 276ms (s: 130, r: 142, is: 0, ir: 0)
PASS (skrrt!):  Tier 31 :       Ran 1072x1072   image with 2698 bodies in 295ms (s: 141, r: 150, is: 0, ir: 0)
PASS (eoo!):    Tier 32 :       Ran 1120x1120   image with 2806 bodies in 319ms (s: 153, r: 163, is: 0, ir: 0)
PASS (yay!):    Tier 33 :       Ran 1152x1152   image with 2918 bodies in 340ms (s: 165, r: 172, is: 0, ir: 0)
PASS (yeee!):   Tier 34 :       Ran 1200x1200   image with 3035 bodies in 371ms (s: 179, r: 191, is: 0, ir: 0)
PASS (ayy!):    Tier 35 :       Ran 1248x1248   image with 3156 bodies in 395ms (s: 193, r: 199, is: 0, ir: 0)
PASS (woot!):   Tier 36 :       Ran 1312x1312   image with 3283 bodies in 434ms (s: 210, r: 223, is: 0, ir: 0)
PASS (woot!):   Tier 37 :       Ran 1360x1360   image with 3414 bodies in 472ms (s: 225, r: 245, is: 0, ir: 0)
PASS (ayy!):    Tier 38 :       Ran 1408x1408   image with 3551 bodies in 504ms (s: 246, r: 257, is: 0, ir: 0)
PASS (woot!):   Tier 39 :       Ran 1472x1472   image with 3693 bodies in 553ms (s: 265, r: 285, is: 0, ir: 0)
PASS (yeee!):   Tier 40 :       Ran 1536x1536   image with 3840 bodies in 597ms (s: 286, r: 309, is: 0, ir: 0)
PASS (eoo!):    Tier 41 :       Ran 1584x1584   image with 3994 bodies in 642ms (s: 309, r: 329, is: 0, ir: 0)
PASS (ayy!):    Tier 42 :       Ran 1648x1648   image with 4154 bodies in 706ms (s: 337, r: 367, is: 0, ir: 0)
PASS (yeee!):   Tier 43 :       Ran 1728x1728   image with 4320 bodies in 755ms (s: 363, r: 390, is: 0, ir: 0)
PASS (eoo!):    Tier 44 :       Ran 1792x1792   image with 4493 bodies in 807ms (s: 393, r: 411, is: 0, ir: 0)
PASS (skrrt!):  Tier 45 :       Ran 1856x1856   image with 4672 bodies in 887ms (s: 427, r: 458, is: 0, ir: 0)
PASS (ayy!):    Tier 46 :       Ran 1936x1936   image with 4859 bodies in 944ms (s: 459, r: 483, is: 0, ir: 0)
FAIL (timeout): Tier 47 :       Ran 2016x2016   image with 5054 bodies in 1014ms (s: 496, r: 517, is: 0, ir: 0) but the cutoff is 1000ms;
used blowthrough 1
FAIL (timeout): Tier 48 :       Ran 2096x2096   image with 5256 bodies in 1125ms (s: 538, r: 585, is: 0, ir: 0) but the cutoff is 1000ms;
used blowthrough 2
FAIL (timeout): Tier 49 :       Ran 2176x2176   image with 5466 bodies in 1204ms (s: 583, r: 620, is: 0, ir: 0) but the cutoff is 1000ms;
used blowthrough 3
Reached tier 46 of 100
Final tier attempted: 49
Time elapsed on final tier: 1204 ms
REMINDER: This is only timing, NOT CORRECTNESS. Ensure you also run correctness tests
Blowthroughs used: 3 / 2
```